

UNIVERSIDAD DE LAS FUERZAS ARMADAS ESPE

DEPARTAMENTO DE CIENCIAS DE LA COMPUTACIÓN
CARRERA DE INGENIERÍA DE SOFTWARE
MODELOS DISCRETOS PARA INGENIERÍA

CALIFICACIÓN:

NRC: 23222

ACTIVIDAD: P3_Ta1_ej1

APELLIDOS Y NOMBRES: Villarreal Salgado Evelyn Haydée

FECHA: 21/07/2025

1. Conversión entre representaciones de grafos

Escriba un programa que acepte como entrada cualquiera de los siguientes elementos:

- una lista de las aristas de una gráfica dada como pares de enteros positivos
- la matriz de adyacencia
- la matriz de incidencia

y proporcione como salida los otros dos.

```
#include <iostream>
```

```
#include <vector>
```

```
using namespace std;
```

```
void imprimirMatriz(const vector<vector<int>>& matriz) {
```

```
    for (const auto& fila : matriz) {
```

```
        for (int val : fila)
```

```
            cout << val << " ";
```

```
        cout << endl;
```

```
    }
```

```
}
```

```
void desdeListaAristas(int n, const vector<pair<int, int>>& aristas) {
```

```
    int m = aristas.size();
```

```
// Construir matriz de adyacencia
```

```
vector<vector<int>> adj(n, vector<int>(n, 0));
```

```
for (auto [u, v] : aristas) {
```

```
    adj[u][v] = 1;
```

```
    adj[v][u] = 1;
```

```
}
```

```

// Construir matriz de incidencia
vector<vector<int>> inc(n, vector<int>(m, 0));

for (int i = 0; i < m; ++i) {
    int u = aristas[i].first;
    int v = aristas[i].second;
    inc[u][i] = 1;
    inc[v][i] = 1;
}

cout << "Matriz de adyacencia:\n";
imprimirMatriz(adj);

cout << "Matriz de incidencia:\n";
imprimirMatriz(inc);
}

void desdeMatrizAdyacencia(const vector<vector<int>>& adj) {
    int n = adj.size();
    vector<pair<int, int>> aristas;
    vector<vector<int>> inc;

    // Obtener lista de aristas (solo u < v para evitar duplicados)
    for (int i = 0; i < n; ++i)
        for (int j = i + 1; j < n; ++j)
            if (adj[i][j])
                aristas.push_back({i, j});

    int m = aristas.size();
    inc.assign(n, vector<int>(m, 0));

    for (int i = 0; i < m; ++i) {
        inc[aristas[i].first][i] = 1;
    }
}

```

```

        inc[aristas[i].second][i] = 1;
    }

    cout << "Lista de aristas:\n";
    for (auto [u, v] : aristas)
        cout << u << " " << v << endl;

    cout << "Matriz de incidencia:\n";
    imprimirMatriz(inc);
}

void desdeMatrizIncidencia(const vector<vector<int>>& inc) {
    int n = inc.size();
    int m = inc[0].size();
    vector<pair<int, int>> aristas;
    vector<vector<int>> adj(n, vector<int>(n, 0));

    // Recorreremos columnas para encontrar pares de vértices
    for (int j = 0; j < m; ++j) {
        int u = -1, v = -1;
        for (int i = 0; i < n; ++i) {
            if (inc[i][j]) {
                if (u == -1) u = i;
                else v = i;
            }
        }
        if (u != -1 && v != -1) {
            aristas.push_back({u, v});
            adj[u][v] = adj[v][u] = 1;
        }
    }
}

```

```

    cout << "Lista de aristas:\n";
    for (auto [u, v] : aristas)
        cout << u << " " << v << endl;

    cout << "Matriz de adyacencia:\n";
    imprimirMatriz(adj);
}

int main() {
    int opcion;
    cout << "Tipo de entrada:\n";
    cout << "1 - Lista de aristas\n";
    cout << "2 - Matriz de adyacencia\n";
    cout << "3 - Matriz de incidencia\n";
    cout << "Opcion: ";
    cin >> opcion;

    if (opcion == 1) {
        int n, m;
        cout << "Numero de vertices: "; cin >> n;
        cout << "Numero de aristas: "; cin >> m;
        vector<pair<int, int>> aristas(m);
        cout << "Ingrese las aristas (pares u v):\n";
        for (int i = 0; i < m; ++i)
            cin >> aristas[i].first >> aristas[i].second;
        desdeListaAristas(n, aristas);
    } else if (opcion == 2) {
        int n;
        cout << "Tamano de la matriz: "; cin >> n;
        vector<vector<int>> adj(n, vector<int>(n));
        cout << "Ingrese la matriz de adyacencia:\n";
        for (int i = 0; i < n; ++i)

```

```

        for (int j = 0; j < n; ++j)
            cin >> adj[i][j];
        desdeMatrizAdyacencia(adj);
    } else if (opcion == 3) {
        int n, m;
        cout << "Numero de vertices: "; cin >> n;
        cout << "Numero de aristas: "; cin >> m;
        vector<vector<int>> inc(n, vector<int>(m));
        cout << "Ingrese la matriz de incidencia:\n";
        for (int i = 0; i < n; ++i)
            for (int j = 0; j < m; ++j)
                cin >> inc[i][j];
        desdeMatrizIncidencia(inc);
    } else {
        cout << "Opción invalida.\n";
    }
    return 0;
}

```

```

Tipo de entrada:
1 - Lista de aristas
2 - Matriz de adyacencia
3 - Matriz de incidencia
Opcion: 1
Numero de vertices: 4
Numero de aristas: 3
Ingrese las aristas (pares u v):
0 1
1 2
2 3
Matriz de adyacencia:
0 1 0 0
1 0 1 0
0 1 0 1
0 0 1 0
Matriz de incidencia:
1 0 0
1 1 0
0 1 1
0 0 1
-----
Process exited after 38.68 seconds with return value 0
Press any key to continue . . . |

```